Exam *DP-700* Study Guide

Implementing Data Engineering Solutions using Microsoft Fabric – OneLake, Lakehouse, Warehouse, Real-Time Intelligence.

A consolidated, scenario-first reference organized around the three official exam domains. Every topic is paired with the decisions and distractors you will actually encounter on the exam.

DOMAIN WEIGHTING

IMPLEMENT & MANAGE · 30–35%

INGEST & TRANSFORM · 30–35%

MONITOR & OPTIMIZE · 30–35%

EXAM CODE	DURATION	QUESTIONS	PASSING SCORE
DP-700	100 min / 120 seat	40 – 60	700 / 1000

00 CONTENTS

What's inside.

Seven chapters covering the exam blueprint, deep topic notes, comparison tables, the traps this exam likes to set, and a twenty-item pre-exam checklist.

01 **Exam Domains & Weightage** p. 03
Blueprint, audience profile, and what the exam tests.

02 **Domain 1 – Implement & Manage** p. 04
Workspace settings, lifecycle, security, orchestration.

03 **Domain 2 – Ingest & Transform** p. 07
Loading patterns, batch + streaming, engines, shortcuts.

04 **Domain 3 – Monitor & Optimize** p. 10
Monitoring, error triage, performance tuning.

05 **Service Comparison Tables** p. 12
Stores, engines, transformation languages.

06 **Common Pitfalls & Distractors** p. 13
Twelve plausible-but-wrong answers the exam loves.

07 **Final Checklist – 20 Must-Knows** p. 14
Night-before review list.

01 OFFICIAL EXAM DOMAINS & WEIGHTAGE

The blueprint.

The DP-700 exam validates your ability to implement data engineering solutions in Microsoft Fabric — ingesting, transforming, securing, and monitoring analytics workloads across OneLake, Lakehouse, Warehouse, Real-Time Intelligence, and Data Factory items.

Audience profile

- Subject matter expertise in data loading patterns, data architectures, and orchestration processes.
- Responsibilities: ingesting and transforming data; securing and managing an analytics solution; monitoring and optimizing it.
- Works with analytics engineers, architects, analysts, and administrators to deploy data engineering solutions.
- Skilled at manipulating and transforming data with **SQL**, **PySpark**, and **KQL** (Kusto Query Language).

Weighting

DOMAIN	WEIGHT	FOCUS
01 Implement & Manage an Analytics Solution	30 – 35%	Workspaces, lifecycle, security
02 Ingest & Transform Data	30 – 35%	Batch + streaming pipelines
03 Monitor & Optimize	30 – 35%	Observability & tuning

EXAM ALERT

Most questions cover GA features; commonly used Preview features may appear. Expect decision-style scenarios — "given this workload, which Fabric item do you pick?" — across nearly every topic area.

Format at a glance

PLATFORMS

Fabric · OneLake · Lakehouse · Warehouse · Real-Time Intelligence

QUESTION TYPES

Multiple choice · Drag-and-drop · Case studies

LANGUAGES

SQL · PySpark · KQL · Power Query (M)

02 DOMAIN 1 · 30 – 35%

Implement & manage the *analytics solution.*

2.1 Configure Fabric workspace settings

- **Spark workspace settings.** Pool size, environment, runtime version, default libraries. Enable high concurrency for shared notebook sessions.
- **Domain workspace settings.** Assign workspaces to domains; domain admins govern discoverability, default settings, and certification rules.
- **OneLake workspace settings.** Enforce case sensitivity, configure caching, enable/disable external access (ADLS gen2 shortcuts, OneLake security).
- **Data workflow workspace settings.** Configure default storage, git branch, managed private endpoints, and deployment rules.

2.2 Lifecycle management

- **Version control.** Git integration with Azure DevOps or GitHub. Each workspace binds to a branch; commit, pull, and resolve conflicts from the workspace UI.
- **Database projects.** Declarative schema for Fabric Warehouse / SQL DB in Fabric, source-controlled alongside notebooks and pipelines.
- **Deployment pipelines.** Promote content through dev → test → prod stages. Use **deployment rules** to swap connection strings, parameters, or data sources per stage.

EXAM ALERT

Know when to use **Git integration** (source control) vs. **deployment pipelines** (stage promotion). They are complementary — Git for change tracking and collaboration, pipelines for controlled rollout between workspaces.

2.3 Security & governance

- **Workspace-level roles.** Admin, Member, Contributor, Viewer — coarse access.
- **Item-level access.** Share individual lakehouses, warehouses, notebooks with specific permissions (read, build, share).
- **Row / column / object / folder-file security.** RLS via security predicates; OLS hides columns; folder and file ACLs in OneLake.
- **Dynamic Data Masking.** Presentation-layer masking on Warehouse columns. Does *not* encrypt.
- **Sensitivity labels.** Apply Microsoft Purview labels to items; labels flow with exported data (downstream protection).
- **Endorsements.** Mark items *Promoted* (by author) or *Certified* (by admin) for trust signals.
- **Workspace logging.** Enable to capture user activity and item events for audit.
- **OneLake security.** Fine-grained access on OneLake tables and folders, surfaced across all engines.

2.4 Orchestrate processes

- **Dataflow Gen2** — low-code Power Query (M) transformations with visual UI. Best for citizen-developer / analyst work.
- **Pipeline** — orchestration engine with Copy, Dataflow, Notebook, and control activities. Best for multi-step flows with parameters.
- **Notebook** — code-first Spark/PySpark/SQL. Best for complex transformations and ML.
- **Schedules & triggers.** Time-based schedules or event-based (file arrival, Reflex/Activator events, Real-Time event grid).
- **Orchestration patterns.** Pipelines invoke notebooks with `baseParameters`; pass outputs back via `mssparkutils.notebook.exit()`; use dynamic expressions `@pipeline()`, `@activity()`, `@variables()`.

Decide: Dataflow · Pipeline · Notebook

TOOL	STRENGTHS	USE WHEN…
Dataflow Gen2	Low-code; visual; reusable M queries.	Analysts build self-service ETL; simple shape & enrich.
Pipeline	Orchestration; parameters; control flow; copy activity.	Multi-step workflow — copy, transform, branch, retry.
Notebook	Code-first; full Spark + PySpark + SQL; libraries.	Complex joins, ML, custom logic, large distributed jobs.

IF → an analyst needs self-service ingestion with minimal code → **Dataflow Gen2** with Power Query.

IF → you must orchestrate "copy → transform → notify" with retries → a **Pipeline** invoking notebooks + copy activities.

IF → a job must run when a file arrives in ADLS → event-based trigger via **Storage Event** on a pipeline.

Security pattern quick-reference

REQUIREMENT	FEATURE	NOTE
Isolate tenants by row	Row-Level Security	Security predicate on user/session context.
Hide sensitive columns	Object / Column-Level Security	`DENY SELECT` on specific columns.
Mask PII at presentation	Dynamic Data Masking	Not encryption — DBAs still see real data.
Restrict access to a folder in OneLake	OneLake folder / file ACLs	Applies across all engines reading the path.
Label & protect downstream	Sensitivity labels (Purview)	Labels follow exports to Excel, PBIX, etc.

02 · CARD LIFECYCLE CARD · GIT + DEPLOYMENT PIPELINES

From notebook to *production.*

STEP 01
Develop
Author notebooks, pipelines, lakehouse schemas in a **dev workspace**.

STEP 02
Source control
Workspace bound to Git branch; commit + pull; resolve conflicts.

STEP 03
Promote
Deployment pipeline pushes items dev → test → prod with rules.

STEP 04
Operate
Monitor Hub, alerts, capacity metrics, schedule + trigger health.

Fabric item types — at a glance

ITEM	PRIMARY PURPOSE	NOTABLE ENGINE
Lakehouse	Unified files + Delta tables over OneLake.	Spark + SQL endpoint
Warehouse	T-SQL data warehouse with multi-table transactions.	T-SQL (Polaris)
KQL Database	Real-time log / telemetry store.	Kusto (KQL)
Eventhouse	Container grouping KQL databases for RTI workloads.	Kusto
Eventstream	No-code streaming ingest / transform / route.	Stream engine
Data pipeline	Orchestration of copy, notebook, dataflow, control.	Data Factory
Notebook	Code-first Spark / PySpark / SQL / Scala / R.	Spark
Dataflow Gen2	Low-code Power Query (M) transformations.	Mashup + Spark
Reflex / Activator	Detect patterns in data, trigger downstream actions.	Reflex

MENTAL MODEL

Lakehouse = files + tables. Warehouse = pure T-SQL with transactions. KQL = time-series + telemetry. Eventstream = *moving* data. Pipeline = *moving* pipelines of work. Match the verb in the question to the Fabric item.

03 DOMAIN 2 · 30 – 35%

Ingest & transform *data.*

3.1 Loading patterns

- **Full load.** Truncate-and-reload the target. Simple, safe — but O(N) every run. Use for small dimensions or when source change tracking is unavailable.
- **Incremental load.** Load only new/changed rows using a **watermark** column (`LastModified` , `ETL_Id`), CDC, or Change Tracking. Use for large fact tables.
- **Streaming load.** Append from Eventstream / Spark Structured Streaming into bronze tables; micro-batch downstream.
- **Prepare for dimensional model.** Conform types; generate surrogate keys; handle SCD Type 1 (overwrite) vs. Type 2 (history rows with effective dates); deduplicate on natural keys.

3.2 Batch ingest & transform

- **Choose a data store.** Lakehouse for files + Delta analytics; Warehouse for pure T-SQL / multi-table transactions; KQL DB for time-series; SQL DB in Fabric for OLTP.
- **Choose a transformation tool.** Dataflow (M) for low-code; Notebook (PySpark/SQL) for complex; KQL for telemetry; T-SQL for set-based operations on Warehouse.
- **Shortcuts.** Virtualize data from ADLS, S3, GCS, Dataverse, or other OneLake items — no data movement, single source of truth.
- **Mirroring.** Near-real-time read-only replica of Azure SQL DB, Cosmos DB, Snowflake, etc. into OneLake as Delta — free of egress.
- **Copy activity (pipeline).** Move data between 100+ connectors with schema mapping, staging, fault tolerance.
- **Continuous integration from OneLake.** Subscribe lakehouse tables to auto-ingest changes into a warehouse or other target.
- **Denormalize, group, aggregate.** Flatten joins for analytics; pre-aggregate into gold tables for BI.
- **Handle duplicates / missing / late data.** `DISTINCT` , `ROW_NUMBER()` + filter; COALESCE defaults; watermarking + side outputs for late events.

IF → you need a live read of Azure SQL without ETL → **Mirroring** into OneLake.

IF → you want to analyze ADLS files as Lakehouse tables without copying → **Shortcut**.

IF → the source only exposes "created_at" and you need to avoid full reloads → **watermark-based incremental load**.

IF → a Type 2 history is required on the Customer dimension → effective-dated rows with `IsCurrent` flag; close prior on change.

03 · CONT. REAL-TIME INTELLIGENCE · STREAMING

3.3 Streaming ingest & transform

- **Streaming engine choice. Eventstream** for no-code ingest + routing (Event Hub, IoT Hub, Kafka, CDC sources). **Spark Structured Streaming** for code-first stateful transforms. **KQL update policy** for in-engine shaping on ingest.
- **Storage choice in RTI. Native** (KQL database) for lowest-latency queries. **Mirrored** for external operational stores. **Shortcut** to reuse existing OneLake tables.
- **Accelerated vs. non-accelerated shortcuts.** Accelerated caches Delta / Iceberg in Kusto columnstore for fast queries; non-accelerated reads on demand.
- **Eventstream processing.** Filter, aggregate, join, manage schema — visual no-code operators; route to Lakehouse, KQL DB, Activator, or custom endpoint.
- **Spark Structured Streaming.** `readStream` → transformations → `writeStream` with checkpointing; trigger modes (default, once, available-now, continuous).
- **KQL.** `.ingest`, update policies, materialized views, continuous export.
- **Windowing.** Tumbling (fixed, non-overlapping), hopping (overlapping), sliding (per-event), session (gap-based). Know the diagrams.

Windowing at a glance

WINDOW	SHAPE	BEST FOR
Tumbling	Fixed, non-overlapping buckets.	5-minute counts, hourly rollups.
Hopping	Fixed length, hops every N seconds (overlapping).	Moving averages, sliding dashboards.
Sliding	Emits on every event within a window size.	Alerts on "last 5 minutes" per event.
Session	Closes after a gap of inactivity.	User sessions, call aggregation.

IF → you need sub-second queries over telemetry with minimal shaping → ingest into a **KQL database** natively.

IF → a downstream team needs the same stream into both Lakehouse *and* KQL → **Eventstream** with two destinations.

IF → you want Delta tables in Lakehouse queryable in Kusto without copy → **accelerated shortcut**.

IF → events may arrive out of order → configure **watermark** + allowed lateness in Structured Streaming.

EXAM ALERT

Eventstream ≠ Eventhouse. Eventstream *moves* events; Eventhouse *stores* KQL databases. Eventstream can land into an Eventhouse — know the direction of data flow.

03 · CONT. PICK THE RIGHT LANGUAGE FOR THE JOB

Power Query (M) · PySpark · SQL · KQL

LANGUAGE	SHINES AT	WHERE IT LIVES	WATCH OUT FOR
Power Query (M)	Low-code shaping; UI-driven transforms.	Dataflow Gen2	Not ideal for very large joins.
PySpark	Distributed compute; ML; complex joins.	Notebooks	Cluster warm-up latency.
SQL (T-SQL)	Set operations; multi-table transactions.	Warehouse, SQL endpoint	T-SQL subset on Lakehouse SQL endpoint is read-mostly.
KQL	Time-series, logs, telemetry analytics.	KQL DB, Eventhouse	Different paradigm — pipe operator, tabular.

Common patterns — quick forms

Deduplicate on natural key

Keep latest per key using a window function.

```
SELECT * FROM (
  SELECT *, ROW_NUMBER() OVER
    (PARTITION BY CustomerId ORDER
BY UpdatedAt DESC) rn
  FROM stg.Customer
) t WHERE rn = 1;
```

Incremental watermark load

Pipeline reads max watermark, copies only newer rows.

```
@{activity('LookupWatermark').output.firstRow.LastLoad}

SELECT * FROM src.Orders
WHERE ModifiedDate > @prevWatermark;
```

Late / duplicate / missing data

- **Duplicates.** `DISTINCT`, `GROUP BY`, or `ROW_NUMBER()` + filter for "latest wins."
- **Missing.** `COALESCE` / `ISNULL` with defaults; or route nulls to a quarantine table.
- **Late-arriving facts.** Use a surrogate key lookup; insert a placeholder dimension row; update fact when dim arrives.
- **Late streaming events.** Set watermark + allowed lateness; route very-late events to a side output for reconciliation.

PRO TIP

When the exam asks "which language for this transformation?" — look for clues: **analyst + UI** → M, **large joins** → PySpark, **multi-table transaction** → T-SQL on Warehouse, **time-series / logs** → KQL.

04 DOMAIN 3 · 30 – 35%

Monitor & optimize *everything.*

4.1 Monitor Fabric items

- **Monitor Hub.** One-stop view of recent activities — pipelines, dataflows, notebooks, semantic model refreshes. Filter by item type, status, user, time.
- **Data ingestion.** Pipeline run history, copy-activity throughput, failed-row counts, source/sink metrics.
- **Data transformation.** Notebook Spark UI — stages, tasks, shuffle; Dataflow refresh logs.
- **Semantic model refresh.** Refresh history, failed partitions, incremental refresh status, DirectLake fallback reasons.
- **Alerts.** Configure in Monitor Hub or via **Reflex/Activator** on KQL/stream events — route to email, Teams, webhook.
- **Capacity metrics.** Fabric Capacity Metrics app — CU seconds, smoothing, throttling events, top consumers.

4.2 Identify & resolve errors

ITEM	COMMON FAILURES	FIRST DIAGNOSTIC
Pipeline	Timeout, 401/403, schema drift, null parameter.	Activity output + error code; run debug with static parameters.
Dataflow	Data-type mismatch, formula error, gateway timeout.	Refresh history error detail + step preview.
Notebook	OOM, skew, library missing, lakehouse path not found.	Spark UI + driver logs; check cluster pool size.
Eventhouse	Ingestion failed, schema mismatch, mapping error.	`.show ingestion failures`, ingestion mappings.
Eventstream	Connection drop, dead-letter growing, routing mismatch.	Runtime logs + destination health.
T-SQL	Deadlock, blocking, bad plan after stats change.	Query insights + `sys.dm_exec_*` DMVs.
Shortcut	Auth expired, source moved, permission denied.	Shortcut properties + underlying source auth.

4.3 Optimize performance

- **Lakehouse table.** Enable `V-Order`; run `OPTIMIZE` (bin-packing) and `VACUUM`; partition by low-cardinality column; `Z-ORDER` on filter columns.
- **Pipeline.** Use parallelism on ForEach; stage large copies via staging blob; enable fault tolerance.
- **Warehouse.** Statistics + automatic; result-set caching; star-schema design; avoid SELECT *; minimize data movement across distributions.
- **Eventstream / Eventhouse.** Right-size throughput units; partition event hubs; use materialized views; set retention & caching policies.
- **Spark.** Right-size pools; broadcast small tables; avoid `collect()`; cache hot DataFrames; enable Adaptive Query Execution.

04 · CONT. TUNING PATTERNS THAT SHOW UP ON THE EXAM

Lakehouse table maintenance

- **V-Order.** Write-time optimization for Delta Parquet that improves read performance on Fabric engines. Enabled by default in Fabric; keep it on.
- **OPTIMIZE.** Compacts small files into larger ones — reduces metadata + IO.
- **Z-ORDER.** Co-locates related rows within files on chosen columns → better data skipping for highly-filtered queries.
- **VACUUM.** Removes files no longer referenced by Delta log after retention window (default 7 days). Reduces storage.
- **Partitioning.** Choose a low-cardinality column (e.g. `EventDate`); avoid high-cardinality (`UserId`) which creates tiny files.

Spark performance levers

Shuffle reduction

- Broadcast small dims with `broadcast()` .
- Pre-partition on join keys.
- Filter and project *before* the join.

Skew handling

- Enable AQE skew join.
- Salt heavy keys.
- Monitor Spark UI for slow tasks.

Cluster & memory

- Right-size driver + executor.
- Cache only DataFrames reused > 2x.
- Prefer Delta over raw Parquet.

Write strategy

- Use `coalesce()` to reduce file count.
- Enable optimized writes.
- Avoid overwrite when merge suffices.

Warehouse query tuning

- **Result-set caching.** Repeat queries return instantly; ensure cache is not invalidated by noise in the query text.
- **Statistics.** Auto-stats are on; manual update helps after large loads.
- **Star-schema design.** Flat fact + conformed dimensions → fewer joins, better parallelism across distributions.
- **Data movement.** Watch for "shuffle" steps in the plan caused by mismatched distribution keys.

PRO TIP

If a Delta table has many tiny files from streaming writes, `OPTIMIZE` is the right answer — not repartitioning in Spark. The exam specifically differentiates between write-time and maintenance-time remedies.

05 SERVICE COMPARISON TABLES

Choose the right *store.*

Fabric data stores compared

FEATURE	LAKEHOUSE	WAREHOUSE	KQL DATABASE	SQL DB IN FABRIC
Primary workload	Files + Delta tables	T-SQL DW	Real-time / telemetry	OLTP
Language	Spark SQL / PySpark	T-SQL	KQL	T-SQL
Writes	FULL	FULL	Append-oriented	FULL OLTP
Multi-table txn	NO	YES	NO	YES
Schema on read	YES	NO	PARTIAL	NO
Shortcuts in	YES	Via OneLake	YES	NO
Best for	Medallion ELT, ML	Star schema BI	Logs, IoT, events	App backend

Ingestion mechanism — which when?

MECHANISM	USE CASE	LATENCY
Copy activity	Scheduled batch from 100+ sources.	Batch
Dataflow Gen2	Low-code ELT with Power Query.	Batch
Shortcut	Virtualize external data — zero copy.	Live (read)
Mirroring	Near-real-time replica of Azure SQL, Cosmos DB, Snowflake.	Near-real-time
Eventstream	No-code streaming ingest + routing.	Seconds
Spark Structured Streaming	Code-first streaming transforms.	Sub-second possible

Transformation language — at a glance

LANGUAGE	STRENGTH	TYPICAL ITEM
Power Query (M)	Low-code, UI-driven shaping.	Dataflow Gen2
PySpark	Distributed compute, ML.	Notebook
T-SQL	Set ops, transactions.	Warehouse, SQL endpoint
KQL	Time-series, logs.	KQL DB, Eventhouse

06 COMMON PITFALLS & DISTRACTOR ANSWERS

Plausible, but *wrong.*

The exam uses plausible-sounding options to test depth of understanding. Below are twelve of the most common traps — read each distractor, then the correction.

01 *"Eventstream and Eventhouse are the same thing."*

WRONG Eventstream **moves** events; Eventhouse **stores** KQL databases. Eventstream can land *into* an Eventhouse.

02 *"Shortcuts copy data into OneLake."*

WRONG Shortcuts **virtualize** data — no copy, no egress. Changes at the source are visible immediately.

03 *"Mirroring is bidirectional."*

WRONG Mirroring is **read-only** in OneLake. Writes happen in the source system (Azure SQL, Cosmos DB, etc.).

04 *"Dynamic Data Masking encrypts data."*

WRONG DDM masks at presentation only. Admins still query real data. For encryption use Always Encrypted / TDE.

05 *"Git integration replaces deployment pipelines."*

WRONG Git tracks changes; **deployment pipelines** promote items across dev/test/prod. Use both.

06 *"Partitioning always improves Lakehouse performance."*

WRONG High-cardinality partitioning creates tiny files and hurts performance. Partition only on low-cardinality filter columns.

07 *"VACUUM is equivalent to OPTIMIZE."*

WRONG OPTIMIZE compacts small files; VACUUM removes obsolete files after retention. Different maintenance steps.

08 *"Dataflow Gen2 is always faster than Notebooks."*

WRONG Low-code is quicker to **build**, not to run. For large joins / complex logic a PySpark notebook is typically faster.

09 *"KQL databases support multi-statement transactions."*

WRONG KQL is append-oriented and analytics-first. For multi-table transactions use **Warehouse** or SQL DB in Fabric.

10 *"Lakehouse SQL endpoint supports full DML."*

WRONG The Lakehouse SQL endpoint is **read-mostly**. For full T-SQL DML use a **Warehouse**.

11 *"Endorsement is the same as sensitivity labeling."*

WRONG Endorsement (Promoted / Certified) signals **trust**. Sensitivity labels enforce **protection**. Different purposes.

12 *"Sliding and tumbling windows are interchangeable."*

WRONG Tumbling is fixed & non-overlapping; sliding emits on every event within a window. Pick by semantics of the metric.

07 NIGHT-BEFORE REVIEW · 20 MUST-KNOW ITEMS

Twenty things you must *know*.

Review the night before. If any item feels unfamiliar, revisit that topic in the guide before going to bed.

- [ ] 01 You can choose between **Dataflow Gen2, Pipeline, Notebook** based on persona + workload.
- [ ] 02 You understand the difference between **Lakehouse, Warehouse, KQL DB, and SQL DB in Fabric**.
- [ ] 03 You can set up **Git integration** and a **deployment pipeline**, and know they are complementary.
- [ ] 04 You can implement **workspace, item, row, column, object, folder, file** access controls.
- [ ] 05 You can apply **sensitivity labels** and **endorsements**, and explain the difference.
- [ ] 06 You can configure **OneLake security** at folder / file level.
- [ ] 07 You know the trade-offs between **full, incremental, and streaming** loading patterns.
- [ ] 08 You can prepare data for a **dimensional model**, including SCD Type 1 and Type 2.
- [ ] 09 You understand **Shortcuts** (zero copy) vs. **Mirroring** (read-only replica).
- [ ] 10 You can ingest via **Copy activity, Dataflow, Eventstream, Spark Structured Streaming**.
- [ ] 11 You can transform with **Power Query (M), PySpark, T-SQL, and KQL**.
- [ ] 12 You can choose **native, mirrored, or shortcut storage** in Real-Time Intelligence.
- [ ] 13 You know the difference between **accelerated and non-accelerated shortcuts**.
- [ ] 14 You can apply **tumbling, hopping, sliding, and session** windows appropriately.
- [ ] 15 You can use the **Monitor Hub**, configure alerts, and read **Capacity Metrics**.
- [ ] 16 You can diagnose errors in **pipelines, dataflows, notebooks, eventhouses, eventstreams, T-SQL, shortcuts**.
- [ ] 17 You can optimize a Lakehouse table with **V-Order, OPTIMIZE, Z-ORDER, VACUUM**.
- [ ] 18 You can optimize **Spark** — broadcast, skew, cache, AQE, cluster sizing.
- [ ] 19 You can optimize a **Warehouse** — statistics, star schema, caching, distribution.
- [ ] 20 You can design end-to-end: ingest → transform → serve → monitor for both batch and streaming.

— GOOD LUCK

Focus on scenario-based *reasoning:* understand not just what each Fabric item does, but *when* and *why* you would choose it over the alternatives.

SOURCE

Microsoft Learn — Study Guide for Exam DP-700, Implementing Data Engineering Solutions Using Microsoft Fabric.

END OF GUIDE · DP-700